



CP5046 Project 1 Report

Comparative Analysis of Software Development Approaches

Full Name: Sun Weichao

Student ID: 14845869

Date: 2026.6.18

Word Count:2186

Project Background

Team Charter

The team consists of four members: Sun Weichao, Wang Bohong, Chen Xianzhe, Liu Haoyang.

1. Sun Weichao — Team Leader / Game Designer / Scrum Coordinator
2. Wang Bohong — Game Designer / Scrum Coordinator
3. Chen Xianzhe — Game Designer / Scrum Coordinator
4. Liu Haoyang — Game Designer / Scrum Coordinator

The project focuses on a learning game designed for kindergarten students. KOPO will be used as the main platform for creating the game activities, while Figma will act as the team's shared workspace for planning, arranging visual layouts, and discussing design ideas.

The members have similar skills and will contribute to the project in comparable ways. These shared responsibilities include simple game design, theme creation, Figma-based collaboration, task discussion, peer review, and progress tracking. Each member is required to complete three game themes.

Analysis of team profile: This team structure fits a lightweight Agile-style project because the group is small, flexible, and able to coordinate without a complex hierarchy. Since everyone has similar responsibilities, the workload is balanced. However, the team still needs to manage possible risks such as inconsistent visual style, unclear learning objectives, and irregular communication. Shared Scrum coordination, a common Figma workspace, and frequent progress checks will help reduce these issues.

Project Description

Project background: Our team is working on a kindergarten game project that supports early learning through play. The planned activities are short and easy for young children to understand, such as sorting objects, comparing items, arranging things in order, and practising basic learning content. KOPO will be the platform for creating the game activities, and Figma will be used as the team's design space for screen planning, layout ideas, and shared visual direction.

Project goals and milestones: The expected outcome is a set of child-friendly game themes with clear interaction, suitable learning content, and a consistent design style. For the Alpha stage, the workload is divided equally: four members will each complete three game levels or themes, making twelve in total. After Alpha, the Beta and Final stages will focus on reviewing the first version, selecting or refining the game direction, applying feedback, and improving the final game themes.

Project contract and interviews: The project contract and interview process will help the team clarify user needs, learning goals, and expected game functions. The feedback gained from these activities will be used to guide further improvements to the game design.

Team responsibilities: All members are expected to create three game themes, work together through Figma, update their progress, give feedback to teammates, and help keep the design direction and learning goals consistent.

Section 1: Software Development Approaches

Historical motivations and assumptions underpinning Waterfall and Agile approaches. Software development approaches describe how software process activities such as specification, development, validation, and evolution are organised. Waterfall and Agile are not simply different project management styles; they are based on different assumptions about requirements, change, feedback, risk, and control. Therefore, comparing them requires examining the historical reasons why each approach emerged and the types of software projects each one assumes.

The Waterfall model represents a plan-based approach. Its historical motivation was to make software development more predictable by separating work into sequential phases, such as requirements specification, design, implementation, testing, and maintenance. This approach assumes that requirements can be understood early, documented clearly, and controlled through a structured plan. It also assumes that progress can be measured against phase completion and that change should be managed carefully because returning to earlier phases can create rework, delay, and additional cost. For projects with stable requirements, high regulation, or strong safety and reliability needs, this kind of disciplined planning can still be valuable. However, Boehm's spiral model already suggested that software development often needs risk-driven learning rather than purely linear progression (Boehm, 1988). Similarly, Boehm and Turner (2003) argue that plan-driven and agile methods should be balanced according to project risk rather than treated as absolute opposites.

Agile approaches emerged partly as a response to the overhead and rigidity of plan-based development. Agile assumes that requirements may be uncertain, customer needs may evolve, and useful knowledge is often discovered through feedback during development. Instead of relying mainly on a complete upfront requirements document, Agile uses progressive planning, lightweight requirements, incremental delivery, frequent validation, and close stakeholder involvement. Highsmith and Cockburn (2001) link Agile development to innovation and adaptability, while Williams and Cockburn (2003) describe Agile as being fundamentally about feedback and change. This means that Agile is not only a faster way of producing software, but a way of learning from users and adapting the product over time.

However, Agile should not be interpreted as an absence of discipline. Migrating to Agile can require major changes in management, culture, communication, and team responsibility (Nerur et al., 2005). Empirical studies also show that Agile practices can be beneficial, but their effectiveness depends on context, team capability, and stakeholder participation (Dybå & Dingsøyr, 2008; Dingsøyr et al., 2012). Conboy (2009) further argues that agility should be understood as the ability to respond to change and create value, not merely as speed or flexibility.

For the kindergarten educational game project, Agile is more suitable than a strict Waterfall model because the learning activities, visual design, and interaction style may need to be refined through feedback. User stories, prototypes, peer review, and incremental game themes can help the team improve the game progressively. However, some plan-based structure is still useful for defining milestones, equal workload, and consistent design. Therefore, the project benefits from an Agile-style approach supported by lightweight planning.

Section 2: Critical analysis of strengths, limitations, and failure

Both Waterfall and Agile have strengths, but neither approach can guarantee software project success. A software project normally succeeds only when it delivers useful outcomes within acceptable time, cost, and quality constraints. Project failure often happens because risks are underestimated, requirements are unclear, stakeholders are poorly engaged, or the team lacks the capability to coordinate technical and managerial work. Wallace et al. (2004) show that software project risk affects project performance through multiple dimensions, while El Emam and Koru (2008) indicate that software project failure remains a continuing issue despite improved development methods.

Waterfall's main strength is control. Because requirements, design, implementation, and testing are organised as separate phases, it can support documentation, predictability, fixed scope, and management visibility. This is useful when requirements are stable and when the project needs strong approval points or formal documentation. However, the same structure becomes a limitation when requirements change. If user needs are misunderstood during specification, the problem may not become visible until design or testing. At that stage, change can cause expensive rework, schedule slippage, and stakeholder conflict. Therefore, Waterfall can fail not because planning is useless, but because it assumes that the most important knowledge can be captured early.

Agile addresses some of these weaknesses through incremental delivery, user stories, frequent validation, rapid feedback, and continuous improvement. These practices can reduce delayed feedback and help teams respond to changing requirements. Agile also encourages stakeholder collaboration, which is important because early prediction of success depends partly on user involvement, realistic expectations, and management support (Procaccino et al., 2002). However, Agile can also fail if it is applied superficially. If a team only uses Agile terminology without real customer involvement, backlog prioritisation, testing discipline, or team coordination, Agile may become "fake Agile" or "Scrumfall" rather than an adaptive process.

A major limitation of Agile is that flexibility can create hidden quality problems if it is not supported by technical discipline. Rapid increments may deliver visible progress, but weak refactoring, incomplete testing, and limited documentation can lead to technical debt. Li et al. (2015) describe technical debt as a significant software engineering management issue, while Behutiye et al. (2017) show that technical debt is particularly relevant in Agile contexts because quick delivery and changing requirements can create long-term maintenance costs. Holvitie et al. (2018) also found that practitioners recognise the connection between Agile practices, technical debt, and process quality.

For the educational game project, Agile is useful because the team can build small game themes, review feedback, and adjust the design. However, the project could still fail if each member works separately without consistent design standards, clear learning objectives, and regular peer review. This shows that software project success depends not only on choosing Agile or Waterfall, but on managing risk, communication, quality, and coordination throughout the project.

Section 3: Engagement with contemporary perspectives (“what comes next”)

A key limitation of process-centric thinking is that it assumes project success can be improved mainly by choosing the correct development process. However, many software problems are not only process problems; they are also problems of understanding users, interpreting requirements, testing assumptions, and learning under uncertainty. Contemporary software engineering therefore needs to move beyond a simple Waterfall versus Agile comparison and combine process models with more human-centred and evidence-based practices.

One important direction is Agile requirements engineering. Traditional plan-based development tries to stabilise requirements early, while Agile treats requirements as something that can be discovered and refined through user stories, customer collaboration, and feedback. Cao and Ramesh (2008) show that Agile requirements engineering involves practices such as face-to-face communication, iterative requirements refinement, and customer prioritisation. This perspective is useful because it recognises that customers may not fully understand or explain their needs at the beginning of a project. For an educational game, this is especially important because children, teachers, and other stakeholders may have different expectations about learning goals, usability, and enjoyment.

Another contemporary practice is software prototyping. Prototyping helps teams make uncertain ideas visible and testable before investing heavily in full implementation. Instead of treating requirements as fixed statements, prototypes allow developers and users to explore possible solutions together. Bjarnason et al. (2023) describe software prototyping as a practice that supports learning, communication, and decision-making across different

development contexts. In the kindergarten educational game project, low-fidelity Figma layouts, simple KOPO game versions, and peer testing can help the team identify whether the activities are understandable, age-appropriate, and engaging.

Design thinking also offers an important perspective beyond traditional process models. It emphasises empathy, problem framing, ideation, prototyping, and testing. Dorst (2011) argues that design thinking is strongly connected to reframing, where the team changes how a problem is understood before trying to solve it. This is valuable because software teams often fail when they solve the wrong problem efficiently. Johansson-Sköldberg et al. (2013) also show that design thinking has developed into a broad approach for understanding complex and ambiguous problems, while Micheli et al. (2019) describe it as a set of practices that can support innovation through user-centred exploration.

Design thinking can also reduce cognitive bias. Liedtka (2015) argues that design thinking can improve innovation outcomes by reducing assumptions and encouraging more evidence-based decision-making. This connects directly with software engineering because developers may assume that users understand an interface, enjoy a feature, or need a function when this has not been tested. Therefore, what comes next after traditional Agile is not necessarily a completely new process model. A more realistic future is a context-sensitive combination of Agile, prototyping, design thinking, user research, and continuous learning.

Section 4: Reflection

Teamwork and coordination are central to managing complexity and uncertainty in software projects. A process model can define activities, but it cannot automatically ensure that team members understand the same goals, communicate effectively, or make good decisions under pressure. This is why Agile and Scrum place strong emphasis on self-organising teams, shared accountability, transparency, inspection, and adaptation.

In the educational game project, the team is small and each member is responsible for creating three game themes. This creates a balanced workload, but it also introduces coordination risks. If members work independently without shared standards, the final project may become inconsistent in visual design, learning objectives, and interaction style. Hoda et al. (2013) explain that self-organising Agile teams often depend on informal roles and shared responsibilities, which means coordination must be actively maintained rather than assumed. Similarly, Moe et al. (2010) show that Scrum teamwork depends on communication, team orientation, and the ability to coordinate work towards common goals.

Scrum concepts can help manage this uncertainty. A Product Backlog can model the team's work by making game themes, design tasks, feedback tasks, and improvement tasks visible. A Sprint Goal can focus the team on a small and achievable outcome, while a Definition of Done can clarify when a game theme is acceptable. Sprint Reviews and peer feedback can

support inspection, and Sprint Retrospectives can help the team adapt its collaboration process. These practices are useful because they convert unclear progress into visible and discussable information.

Visual planning is also important because it helps the team make abstract ideas easier to understand. In this project, Figma layouts, simple prototypes, user stories, image materials, and game flow plans can act as lightweight representations of the final game. They allow the team to discuss design consistency, usability, and learning goals before final implementation. Strode et al. (2022) argue that Agile teamwork effectiveness depends on factors such as shared leadership, adaptability, peer feedback, communication, mutual trust, and shared mental models. This supports the idea that visual planning is not only a design activity, but also a coordination tool.

More broadly, contemporary software engineering is moving towards continuous learning and continuous coordination. Fitzgerald and Stol (2017) describe continuous software engineering as an approach that extends Agile thinking through faster feedback and ongoing improvement. DevOps also highlights collaboration, automation, and continuous delivery across software development activities (Leite et al., 2019). Although this educational game project is small, the same principle applies: successful software work depends on regular feedback, shared understanding, and disciplined adaptation, not only on selecting a development method.

Reference (Use APA style)

1. Boehm, B. W. (1988). A spiral model of software development and enhancement. *Computer*, 21(5), 61–72. <https://doi.org/10.1109/2.59>
2. Boehm, B., & Turner, R. (2003). Using risk to balance agile and plan-driven methods. *Computer* (Long Beach, Calif.), 36(6), 57–66. <https://doi.org/10.1109/MC.2003.1204376>
3. Nerur, S., Mahapatra, R., & Mangalaraj, G. (2005, May). Challenges of migrating to agile methodologies. *Communications of the ACM*, 48(5), 72–78. <https://doi.org/10.1145/1060710.1060712>
4. Highsmith, J., & Cockburn, A. (2001). Agile software development: the business of innovation. *Computer* (Long Beach, Calif.), 34(9), 120–127. <https://doi.org/10.1109/2.947100>
5. Williams, L., & Cockburn, A. (2003). Agile software development: it's about feedback and change. *Computer* (Long Beach, Calif.), 36(6), 39–43. <https://doi.org/10.1109/MC.2003.1204373>
6. Dybå, T., & Dingsøyr, T. (2008). Empirical studies of agile software development: A systematic review. *Information and Software Technology*, 50(9), 833–859. <https://doi.org/10.1016/j.infsof.2008.01.006>

7. Dingsøyr, T., Nerur, S., Balijepally, V., & Moe, N. B. (2012). A decade of agile methodologies: Towards explaining agile software development. *The Journal of Systems and Software*, 85(6), 1213–1221. <https://doi.org/10.1016/j.jss.2012.02.033>
8. Conboy, K. (2009). Agility from First Principles: Reconstructing the Concept of Agility in Information Systems Development. *Information Systems Research*, 20(3), 329–354. <https://doi.org/10.1287/isre.1090.0236>
9. Wallace, L., Keil, M., & Rai, A. (2004). How Software Project Risk Affects Project Performance: An Investigation of the Dimensions of Risk and an Exploratory Model. *Decision Sciences*, 35(2), 289–321. <https://doi.org/10.1111/j.00117315.2004.02059.x>
10. El Emam, K., & Koru, A. G. (2008). A Replicated Survey of IT Software Project Failures. *IEEE Software*, 25(5), 84–90. <https://doi.org/10.1109/MS.2008.107>
11. Drew Procaccino, J., Verner, J. M., Overmyer, S. P., & Darter, M. E. (2002). Case study: factors for early prediction of software development success. *Information and Software Technology*, 44(1), 53–62. [https://doi.org/10.1016/S0950-5849\(01\)00217-8](https://doi.org/10.1016/S0950-5849(01)00217-8)
12. Li, Z., Avgeriou, P., & Liang, P. (2015). A systematic mapping study on technical debt and its management. *The Journal of Systems and Software*, 101, 193–220. <https://doi.org/10.1016/j.jss.2014.12.027>
13. Behutiye, W. N., Rodríguez, P., Oivo, M., & Tosun, A. (2017). Analyzing the concept of technical debt in the context of agile software development: A systematic literature review. *Information and Software Technology*, 82, 139–158. <https://doi.org/10.1016/j.infsof.2016.10.004>
14. Holvitie, J., Licorish, S. A., Spínola, R. O., Hyrynsalmi, S., MacDonell, S. G., Mendes, T. S., Buchan, J., & Leppänen, V. (2018). Technical debt and agile software development practices and processes: An industry practitioner survey. *Information and Software Technology*, 96, 141–160. <https://doi.org/10.1016/j.infsof.2017.11.015>
15. Lan Cao, & Ramesh, B. (2008). Agile Requirements Engineering Practices: An Empirical Study. *IEEE Software*, 25(1), 60–67. <https://doi.org/10.1109/MS.2008.1>
16. Bjarnason, E., Lang, F., & Mjöberg, A. (2023). An empirically based model of software prototyping: a mapping study and a multi-case study. *Empirical Software Engineering : An International Journal*, 28(5), Article 115. <https://doi.org/10.1007/s10664-023-10331-w>
17. Dorst, K. (2011). The core of ‘design thinking’ and its application. *Design Studies*, 32(6), 521–532. <https://doi.org/10.1016/j.destud.2011.07.006>
18. Johansson-Sköldberg, U., Woodilla, J., & Çetinkaya, M. (2013). Design Thinking: Past, Present and Possible Futures. *Creativity and Innovation Management*, 22(2), 121–146. <https://doi.org/10.1111/caim.12023>
19. Liedtka, J. (2015). Perspective: Linking Design Thinking with Innovation Outcomes through Cognitive Bias Reduction. *The Journal of Product Innovation Management*, 32(6), 925–938. <https://doi.org/10.1111/jpim.12163>
20. Micheli, P., Wilner, S. J. S., Bhatti, S. H., Mura, M., & Beverland, M. B. (2019). Doing Design Thinking: Conceptual Review, Synthesis, and Research Agenda. *The Journal of Product Innovation Management*, 36(2), 124–148. <https://doi.org/10.1111/jpim.12466>
21. Hoda, R., Noble, J., & Marshall, S. (2013). Self-Organizing Roles on Agile Software Development Teams. *IEEE Transactions on Software Engineering*, 39(3), 422–444. <https://doi.org/10.1109/TSE.2012.30>

22. Moe, N. B., Dingsøyr, T., & Dybå, T. (2010). A teamwork model for understanding an agile team: A case study of a Scrum project. *Information and Software Technology*, 52(5), 480–491. <https://doi.org/10.1016/j.infsof.2009.11.004>
23. Strode, D., Dingsøyr, T., & Lindsjorn, Y. (2022). A teamwork effectiveness model for agile software development. *Empirical Software Engineering : An International Journal*, 27(2), Article 56. <https://doi.org/10.1007/s10664-021-10115-0>
24. Fitzgerald, B., & Stol, K.-J. (2017). Continuous software engineering: A roadmap and agenda. *The Journal of Systems and Software*, 123, 176–189. <https://doi.org/10.1016/j.jss.2015.06.063>
25. Leite, L., Rocha, C., Kon, F., Milojicic, D., & Meirelles, P. (2020). A Survey of DevOps Concepts and Challenges. *ACM Computing Surveys*, 52(6), 1–35. <https://doi.org/10.1145/3359981>